

Transitions: DS + Algorithms

Recursive
algorithm



First:

Questions on older reading?

Plus worksheet...

Multiplication: Can we do better?

Claim:

$$x \cdot y = \begin{cases} 0 & \text{if } x = 0 \\ \lfloor x/2 \rfloor \cdot (y + y) & \text{if } x \text{ is even} \\ \lfloor x/2 \rfloor \cdot (y + y) + y & \text{if } x \text{ is odd} \end{cases}$$

Why? Proof by cases:

if x is even:

if x is odd:

How to make an algorithm?

Note: historical
name!

Not mine--

PEASANTMULTIPLY(x, y):

if $x = 0$

return 0

else

$x' \leftarrow \lfloor x/2 \rfloor$

$y' \leftarrow y + y$

$prod \leftarrow \text{PEASANTMULTIPLY}(x', y')$ *⟨⟨Recurse!⟩⟩*

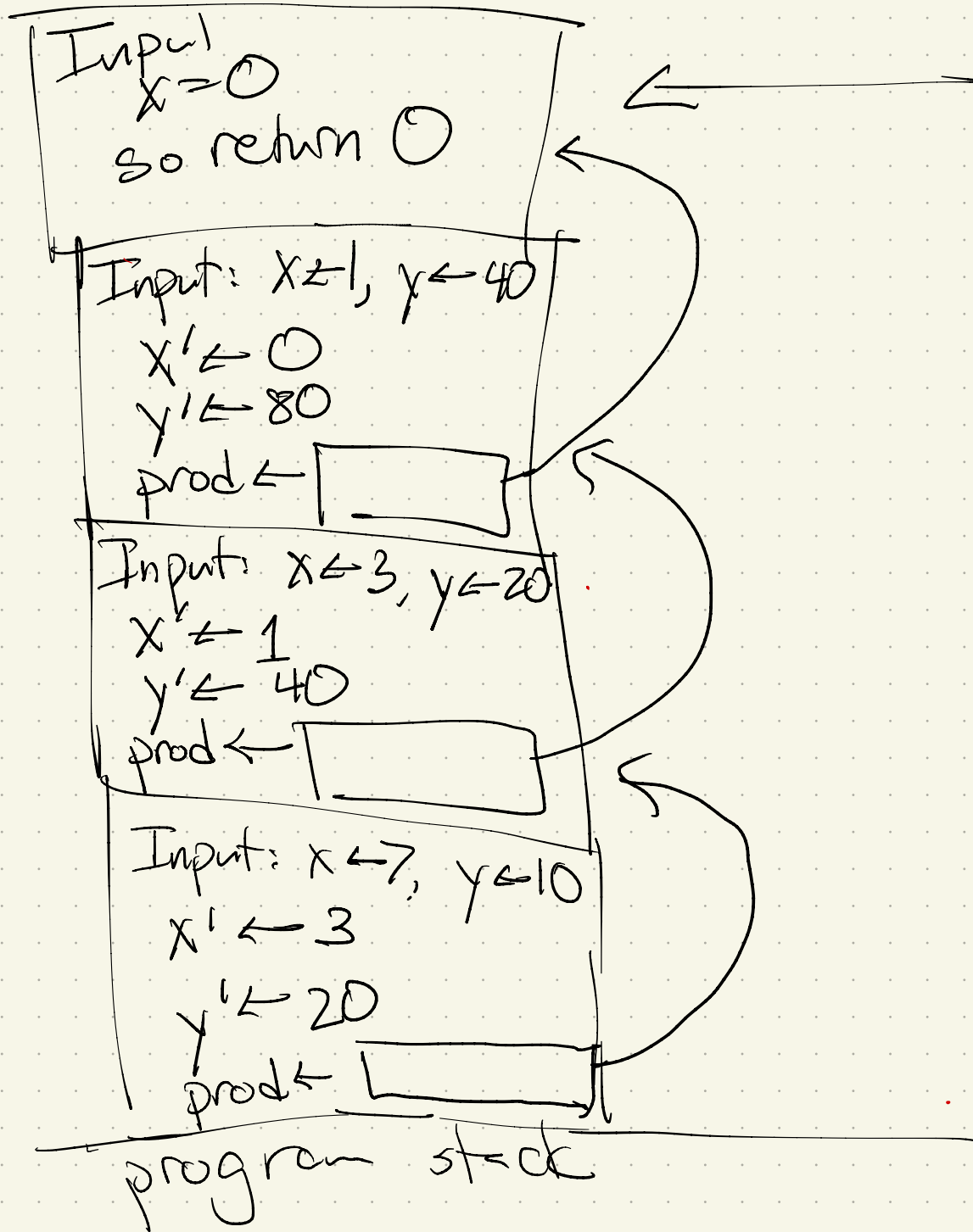
if x is odd

$prod \leftarrow prod + y$

return $prod$

Suppose we call $PM(7, 10)$:

More carefully: FM(7, 10)



Base
case!

PEASANTMULTIPLY(x, y):

if $x = 0$

return 0

else

$x' \leftarrow \lfloor x/2 \rfloor$

$y' \leftarrow y + y$

$prod \leftarrow \text{PEASANTMULTIPLY}(x', y')$

⟨⟨Recurse!⟩⟩

if x is odd

$prod \leftarrow prod + y$

return $prod$

Correctness

Induction on x , for any y :

Base case: $x=0$

IH: For values $x' < x$, the algorithm returns $x' \cdot y$.

IS: Consider $x \cdot y$, $x > 0$:

If x is even: algorithm computes $\lfloor \frac{x}{2} \rfloor \cdot (y+y)$ correctly (by IH)

$$\text{and } \lfloor \frac{x}{2} \rfloor \cdot (y+y) = x \cdot y$$

If odd: algorithm computes

How can we calculate runtime?

PEASANTMULTIPLY(x, y):

if $x = 0$

return 0

else

$x' \leftarrow \lfloor x/2 \rfloor$

$y' \leftarrow y + y$

$prod \leftarrow \text{PEASANTMULTIPLY}(x', y')$ *⟨⟨Recurse!⟩⟩*

if x is odd

$prod \leftarrow prod + y$

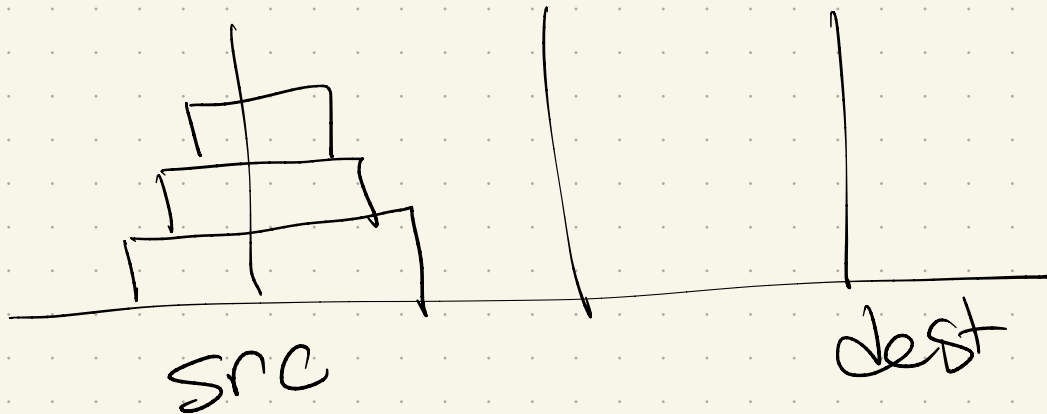
return $prod$

Towers of Hanoi:

```
HANOI( $n, src, dst, tmp$ ):  
  if  $n > 0$   
    HANOI( $n - 1, src, tmp, dst$ )  «Recurse!»  
    move disk  $n$  from  $src$  to  $dst$   
    HANOI( $n - 1, tmp, dst, src$ )  «Recurse!»
```

Figure 1.4. A recursive algorithm to solve the Tower of Hanoi

What is this:



Claim: Algorithm works

Proof of Correctness.
(induction on #disks)

Base case:

IH:

IS:

HANOI(n, src, dst, tmp):

if $n > 0$

HANOI($n - 1, src, tmp, dst$) *⟨⟨Recurse!⟩⟩*

move disk n from src to dst

HANOI($n - 1, tmp, dst, src$) *⟨⟨Recurse!⟩⟩*

Figure 1.4. A recursive algorithm to solve the Tower of Hanoi

Runtime?

Count # times
the disk moves!

HANOI(n, src, dst, tmp):

if $n > 0$

HANOI($n - 1, src, tmp, dst$) *«Recurse!»*

move disk n from src to dst

HANOI($n - 1, tmp, dst, src$) *«Recurse!»*

Figure 1.4. A recursive algorithm to solve the Tower of Hanoi

Solving recurrences (after MergeSort)